Integrated Code Refactoring Report (Final Version)

Introduction

This document presents a comprehensive **code refactoring analysis** performed on several core services of the prototype application. The primary objective of the refactor was to transform a codebase that was *functionally correct but structurally fragile* into one that is **clean, modular, testable, and maintainable**, while fully preserving existing functionality.

The refactoring process focused on identifying classical *code smells* and systematically applying well-known refactoring techniques and object-oriented design principles.

The services analyzed are: - AuthService - GradeService - OptionalCourseRequirementService - TransferRequestService

Each section includes: - Identified problems (Before refactoring) - Applied refactoring techniques - Resulting structure (After refactoring) - Concrete **Before vs After code comparisons**

---

1. AuthService Refactoring

Problems Identified (Before)

1. **Long Method**
2. The `authenticate()` method exceeded 40 lines and handled validation, user creation, token generation, and DTO construction.
3. **Duplicate Code**
4. User and Student creation logic appeared multiple times.
5. Student-specific field updates were duplicated.
6. **Primitive Obsession**
7. Heavy use of `instanceof` and explicit casting instead of polymorphism.
8. **Hidden Conditional Logic**
9. Role-based behavior implemented using chained `if-else` conditions.

Before Refactoring (Code Example)

```
public LoginResponseDTO authenticate(String email, String password) {
    UaicApiResponseDTO apiResponse = uaicApiService.authenticateUser(email,
```

```
password);

    if (apiResponse == null || !apiResponse.isAuthenticated()) {
        throw new RuntimeException("Invalid credentials");
    }

    User user =
userRepository.findByEmail(apiResponse.getEmail()).orElse(null);

    if (user == null) {
        if (apiResponse.getRole() == UserRole.STUDENT) {
            user = Student.builder()
                    .email(apiResponse.getEmail())
                    .firstName(apiResponse.getFirstName())
                    .lastName(apiResponse.getLastName())
                    .matriculationNumber(apiResponse.getMatriculationNumber())
                    .build();
        } else {
            user = User.builder()
                    .email(apiResponse.getEmail())
                    .firstName(apiResponse.getFirstName())
                    .lastName(apiResponse.getLastName())
                    .build();
        }
        userRepository.save(user);
    }

    if (user instanceof Student) {
        Student student = (Student) user;
        student.setMatriculationNumber(apiResponse.getMatriculationNumber());
    }

    String token = jwtService.generateToken(user);
    return new LoginResponseDTO(token, user.getEmail());
}
```

Refactoring Techniques Applied

- Extract Method
- Extract Class (`UserFactory`)
- Replace Type Code with Strategy
- Consolidate Conditional Expressions

After Refactoring (Code Example)

```
public LoginResponseDTO authenticate(String email, String password) {
    UaicApiResponseDTO apiResponse = uaicApiService.authenticateUser(email,
password);
    validateApiResponse(apiResponse);
```

```
    User user = findOrCreateUser(apiResponse);
    String token = generateUserToken(user);

    return buildLoginResponse(user, token);
}
```

```
@Component
class UserFactory {
    public User createUser(UaicApiResponseDTO apiResponse) {
        return switch (apiResponse.getRole()) {
            case STUDENT -> createStudent(apiResponse);
            default -> createBaseUser(apiResponse);
        };
    }
}
```

Result

- Authentication logic is now clearly orchestrated.
- User creation is centralized and polymorphic.
- Code duplication is eliminated.
- Each method has a single, well-defined responsibility.

---

2. GradeService Refactoring

Problems Identified (Before)

1. **Giant Method**
2. `uploadCsv()` exceeded 100 lines.
3. **Data Clumps**
4. CSV row data and statistics were passed as separate primitive values.
5. **Primitive Obsession**
6. Heavy use of arrays, maps, and counters instead of domain objects.
7. **Duplicate Logic**
8. CSV parsing and validation logic repeated across the method.

Before Refactoring (Code Example)

```
@Transactional
public GradeUploadResultDTO uploadCsv(MultipartFile file, boolean overwrite)
```

```
{
    if (file.isEmpty()) {
        throw new RuntimeException("Empty file");
    }

    List<String> lines = readAllLines(file);
    String[] headers = lines.get(0).split(",");

    int inserted = 0, updated = 0, skipped = 0;

    for (int i = 1; i < lines.size(); i++) {
        String[] cols = lines.get(i).split(",");
        // Business logic, validation, persistence
    }

    return new GradeUploadResultDTO(inserted, updated, skipped);
}
```

Refactoring Techniques Applied

- Major Extract Method
- Extract Class (`CsvData`, `CsvRow`, `UploadStatistics`)
- Introduce Parameter Object
- Replace Temp with Query

After Refactoring (Code Example)

```
@Transactional
public GradeUploadResultDTO uploadCsv(MultipartFile file, boolean overwrite)
{
    validateFile(file);

    CsvData csvData = csvParser.parse(file);
    validateCsvHeaders(csvData.getHeaders());

    Map<String, CourseBase> courses = loadCourses(csvData);
    UploadStatistics stats = processCsvRows(csvData, courses, overwrite);

    return buildResult(stats);
}
```

```
class UploadStatistics {
    private int inserted;
    private int updated;
    private int skipped;

    public void incrementInserted() { inserted++; }
}
```

Result

- The upload flow is split into clear phases.
- Domain concepts are explicitly modeled.
- Method complexity is drastically reduced.

---

3. OptionalCourseRequirementService Refactoring

Problems Identified (Before)

- Duplicate percentage validation logic.
- Identical DTO construction in multiple methods.
- Long parameter lists.
- Service class tightly coupled to repositories.

Refactoring Techniques Applied

- Extract Class (`RequirementDtoMapper`, `CourseValidationService`)
- Move Method
- Preserve Whole Object
- Remove Duplicate Code

Before Refactoring (Code Example)

```
if (dto.getPercentage() < 0 || dto.getPercentage() > 100) {
    throw new BadRequestException("Percentage must be between 0 and 100");
}
```

After Refactoring (Code Example)

```
@Component
class CourseValidationService {
    public void validateRequirement(OptionalCourseRequirementDTO dto) {
        if (dto.getPercentage() < 0 || dto.getPercentage() > 100) {
            throw new BadRequestException("Percentage must be between 0 and
100");
        }
    }
}
```

```
@Component
class RequirementDtoMapper {
    public OptionalCourseRequirementResponseDTO
toDto(OptionalCourseRequirement req) {
        return OptionalCourseRequirementResponseDTO.builder()
                .id(req.getId())
                .mandatoryId(req.getMandatoryCourse().getId())
```

```
                .mandatoryName(req.getMandatoryCourse().getName())
                .percentage(req.getPercentage())
                .build();
    }
}
```

Result

- Validation and mapping logic is centralized.
- Services are significantly simplified.

---

4. TransferRequestService Refactoring

Problems Identified (Before)

- Long methods combining multiple responsibilities.
- Duplicate status validation logic.
- Message chains and data clumps.

Refactoring Techniques Applied

- Extract Class (`TransferValidator`, `TransferProcessor`, `TransferDtoMapper`)
- Introduce Parameter Object
- Consolidate Duplicate Conditional Fragments
- Hide Delegate

Before Refactoring (Code Example)

```
if (tr.getStatus() != TransferStatus.PENDING) {
    throw new BadRequestException("Only PENDING allowed");
}
```

After Refactoring (Code Example)

```
@Component
class TransferValidator {
    public void validatePendingStatus(TransferRequest tr) {
        if (tr.getStatus() != TransferStatus.PENDING) {
            throw new BadRequestException("Only requests in PENDING can be
processed");
        }
    }
}
```

```
@Transactional
public TransferRequestResponseDTO approve(Long id) {
    TransferRequest tr = findTransferRequestOrThrow(id);
```

```
    validator.validatePendingStatus(tr);
    processor.executeTransfer(tr);
    tr.setStatus(TransferStatus.APPROVED);
    return mapper.toDto(tr);
}
```

Result

- Service acts as an orchestration layer.
- Business logic, validation, and mapping are fully decoupled.

General Design Principles Applied

1. **Single Responsibility Principle (SRP)**
2. **Don't Repeat Yourself (DRY)**
3. **Law of Demeter**
4. **Separation of Concerns**
5. **Composition over Inheritance**

Final Benefits Achieved

- Substantially improved testability through small, isolated units.
- Reduced maintenance risk and change impact.
- Lower cognitive load for new developers.
- Clear, scalable architecture ready for future extensions.

Conclusion

This refactoring effort transformed the codebase from a *"working but fragile"* implementation into a **robust, well-architected, and maintainable system**. The resulting design adheres to established software engineering best practices and significantly improves long-term sustainability and scalability.

Role of AI Assistance

Artificial Intelligence tools were used as a **supporting aid** during the refactoring process. AI assisted in identifying code smells, suggesting refactoring techniques, and highlighting architectural improvements. All final design decisions and implementations were reviewed and validated by the developer.